Established – 1961 Subject: OS DBMS

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE

ULHASNAGAR – 421 003

CERTIFICATE

This is to certify that Mr./Ms. Ishita Sharma of S.Y. Information Technology (SYIT) Roll No. 2542042 has satisfactorily completed the Open Source DataBase Management System Mini Project entitled Water Supply Complaint Management System During the academic year 2025 – 2026, as a part of the practical requirement. The project work is found to be satisfactory and is approved for submission.

PROF. INCHARGE **HEAD OF DEPT**

_____ _____

INDEX

Sr. No.	Chapter Title	PAGE NO
1	Introduction	3
2	Problem Definition	4
3	Objectives of the Project	5
4	Scope of the Project	6
5	Requirement Specification	7
6	System Design	8
7	Database Design	9
8	UML Diagrams	12
9	SQL Implementation	16
10	System Testing and Result	20
11	Security, Backup and Recovery	24
12	Future Scope and Conclusion	25
13	References	26
14	Glossary	26
15	Appendix	27

1. INTRODUCTION

A Database Management System (DBMS) is a software system that enables users to store, organize, retrieve, and manage data in a structured and efficient manner. It allows data to be stored in tables with relationships between them, ensuring consistency and reducing redundancy. DBMS plays a crucial role in modern information systems where large volumes of data need to be handled securely and efficiently.

The Water Supply Complaint Management System is a database-driven application designed to manage complaints related to water supply services. In many localities, water complaints such as leakage, low pressure, irregular supply, and contaminated water are recorded manually in registers. Manual methods often result in misplaced records, duplication of complaints, delayed responses, and difficulty in tracking complaint status.

The proposed system replaces manual record-keeping with a structured relational database using MySQL. The system stores area information, user details, complaint records, and complaint status updates in separate but related tables. Each complaint is uniquely identified, and its status can be tracked from registration to resolution.

MySQL is chosen as the database platform because it is open-source, reliable, secure, and widely used in academic and industrial applications. It supports SQL queries, constraints, transactions, and multi-user access control. By implementing this system, water supply departments can maintain accurate records, improve response time, and generate meaningful reports for analysis.

This project demonstrates how database concepts such as primary keys, foreign keys, joins, transactions, constraints, and views can be applied to solve real-world administrative problems effectively.

2. PROBLEM DEFINITION

In many municipal corporations and residential societies, water complaints are handled manually. Complaints are written in physical registers or stored in unorganized spreadsheets. As the number of complaints increases, managing these records becomes difficult.

The major issues in the existing system are:

Data Redundancy – The same complaint may be recorded multiple times, leading to duplication.
Data Inconsistency – Updates made in one place may not be reflected elsewhere.
Difficulty in Tracking – It becomes difficult to check whether a complaint is pending or resolved.
Slow Retrieval – Searching past complaints takes time.
Lack of Data Integrity – Complaints may be recorded without linking them to proper users or areas.
Risk of Data Loss – Paper records can be damaged, misplaced, or lost.
No Proper Reporting – Manual systems cannot easily generate analytical reports.
These problems indicate the need for a structured and centralized database system that ensures accuracy, reliability, and easy access to complaint information.

3. OBJECTIVES OF THE PROJECT

The primary objective of the Water Supply Complaint Management System is to design and implement a structured database system using MySQL for managing water supply complaints efficiently.

The specific objectives of the project are:

• To develop a relational database structure for storing complaint information.
• To maintain user and area details in separate but related tables.
• To reduce data redundancy and duplication using constraints.
• To implement SQL queries for data insertion, updating, deletion, and retrieval.
• To enable tracking of complaint status (Pending / Resolved).
• To implement transaction control for complaint registration and updates.
• To generate reports using SQL queries such as GROUP BY and JOIN.
• To gain practical knowledge of MySQL and database design concepts

4. SCOPE OF THE PROJECT

The scope of the Water Supply Complaint Management System is limited to database design and implementation using MySQL. The system is designed for small-scale municipal zones, housing societies, or educational demonstration purposes.

Users of the System:

Admin – Responsible for managing areas, viewing complaints, updating complaint status, and generating reports.
User – Registers complaints and can check complaint status.

Application Areas:

• Municipal water departments
• Residential societies
• Educational projects
• Small administrative offices

Limitations:

• No graphical web interface included.
• No SMS or email notification system.
• Designed primarily for backend database learning purposes.

Despite these limitations, the system provides a strong foundation for real-world complaint management systems.

5. REQUIREMENT SPECIFICATION

5.1 Hardware Requirements

The following describes the hardware and software requirements necessary for developing and implementing the Water Supply Complaint Management System.

- **Computer / Laptop:**
  A standard desktop computer or laptop capable of running MySQL.

- **Minimum RAM:**
  At least **4 GB RAM** is required for smooth functioning of MySQL Server.

- **Storage:**
  Minimum 10–20 GB Free Storage required for storing database files and backups.

5.2 Software Requirements

The following software tools are required for the development and execution of the Water Supply Complaint Management System.

Software	Purpose
MySQL Server	Used to create, store, manage, and secure the complaint database.
MySQL Workbench	Provides a graphical interface for writing, executing, and managing SQL queries
Operating System (Windows / Linux)	Platform required to install and run MySQL Server and related tools
SQL	Query language used to perform database operations such as data insertion, updating, deletion, and retrieval

6. SYSTEM DESIGN

System design defines how the Water Supply Complaint Management System operates and how different components interact with each other.

6.1 System Architecture (Conceptual)

The Water Supply Complaint Management System follows a database-centered architecture where users interact with the system through SQL queries executed on a MySQL database. The architecture is designed to ensure proper data flow, data integrity, and reliable transaction handling.

In this system, administrators or complaint-handling staff perform operations such as registering complaints, updating complaint status, viewing complaint records, and generating reports using SQL commands through MySQL Workbench. All interactions are carried out directly at the database level.

The MySQL Server acts as the core component of the system. It stores data in structured tables such as Area, User_Details, and Complaint. The server enforces constraints like primary keys, foreign keys, unique constraints, and not-null conditions to maintain data integrity and prevent duplication or invalid entries.

Transaction management is implemented to ensure consistency during operations involving multiple steps, such as complaint registration. If any error occurs during execution, the transaction is rolled back to prevent partial data storage. MySQL also manages simultaneous user requests using locking and isolation mechanisms to maintain stability.

Overall, the system architecture ensures efficient complaint management while demonstrating practical implementation of DBMS concepts such as query execution, constraints, transaction control, and concurrency handling.

7. DATABASE DESIGN

The database design of the Water Supply Complaint Management System is based on the relational model. The database is divided into multiple related tables to store complaint data in a structured and organized manner. Each table represents a real-world entity, and relationships between tables are maintained using foreign keys to ensure data integrity and consistency.

7.1 Entity Description

Area Table
The Area table stores information related to different localities or service zones. It contains details such as area name. Each area is uniquely identified using an area_id. This table helps categorize complaints based on geographical location and ensures proper area-wise tracking of complaints.

User_Details Table
The User_Details table stores information about users who register complaints. It includes details such as user name, contact number, address, and associated area. Each user is uniquely identified using a user_id. This table helps maintain accurate complainant records and prevents duplicate user entries.

Complaint Table
The Complaint table stores details of water-related complaints registered by users. Each complaint is linked to a specific user using a foreign key. It records complaint type, complaint date, and complaint status. This table helps track multiple complaints submitted by different users and ensures proper monitoring of complaint resolution.

7.2 Table Structure

Area Table

Attribute	Data Type
area_id	INT (Primary Key, Auto Increment)
area_name	VARCHAR(50)

User_Details Table

Attribute	Data Type
user_id	INT (Primary Key, Auto Increment)
user_name	VARCHAR(50)
phone_no	VARCHAR(15)
address	VARCHAR(100)
area_id	INT (Foreign Key)

Complaint Table

Attribute	**Data Type**
complaint_id	INT (Primary Key, Auto Increment)
user_id	INT (Foreign Key)
complaint_type	VARCHAR(100)
complaint_date	DATE
status	VARCHAR(20)

7.3 Constraints Used

Constraints are used in the Water Supply Complaint Management System to maintain data accuracy, consistency, and integrity. They ensure that only valid and meaningful data is stored in the database. The following constraints are used extensively in this project:

PRIMARY KEY

The PRIMARY KEY constraint is used to uniquely identify each record in a table. In this project, primary keys are used in all major tables such as Area, User_Details, and Complaint. Each table has a unique identifier like area_id, user_id, or complaint_id. This ensures that every record can be uniquely accessed and prevents duplicate entries within the same table.

FOREIGN KEY

The FOREIGN KEY constraint is used to establish relationships between different tables. In this project, foreign keys are used to link user records with complaints and area records with users. This constraint ensures that records in one table always refer to valid records in another table. It prevents invalid or orphan records and maintains logical connections between complaint data.

NOT NULL

The NOT NULL constraint is used to ensure that important fields always contain values. In this project, attributes such as user_name, phone_no, complaint_type, and status are defined as NOT NULL. This ensures that incomplete or meaningless records are not stored in the database and that essential information is always available.

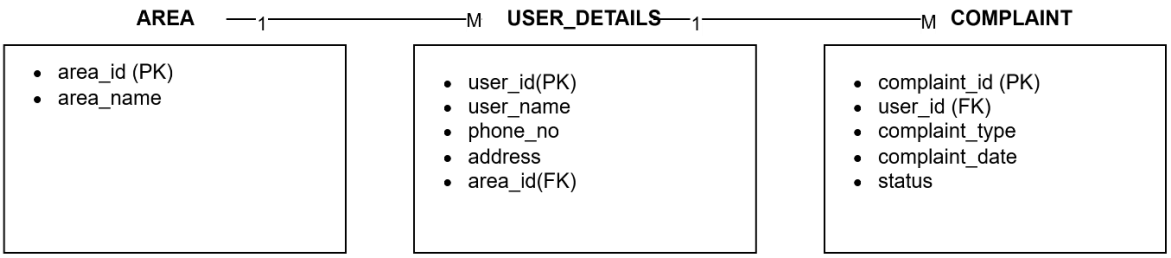
UNIQUE

The UNIQUE constraint is used to prevent duplicate entries in specific fields. In this project, it is mainly used for phone numbers to ensure that the same user contact number is not stored multiple times in the database. This reduces redundancy and maintains data accuracy.

8. UML DIAGRAMS

In this project, UML diagrams such as ER Diagram, Use Case Diagram, Activity Diagram, and Sequence Diagram are used to represent the database structure, user interaction, workflow, and query execution flow.

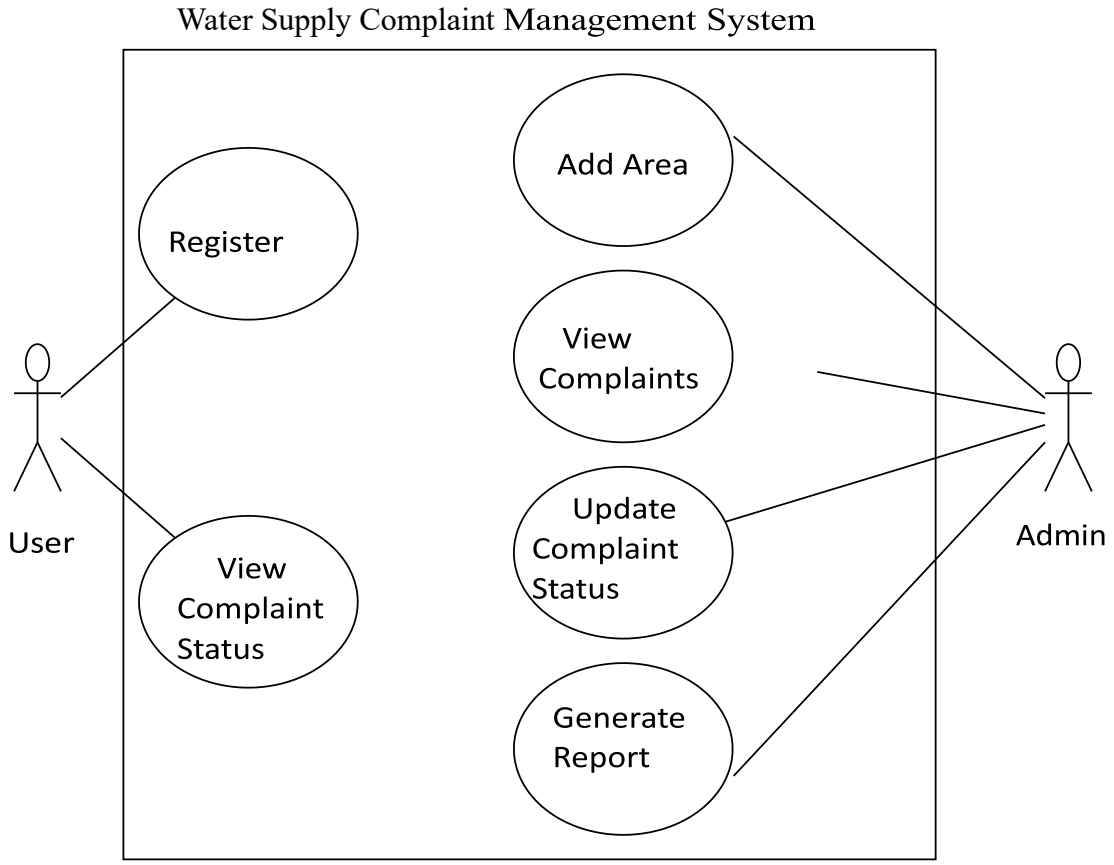
8.1 ER Diagram of Water Supply Complaint Management System



• Entities: Area, User_Details, Complaint
• Relationships maintained using foreign keys

The ER Diagram represents the main entities of the Water Supply Complaint Management System and shows how they are related. It explains how users are linked to areas and how complaints are linked to users using primary and foreign keys. This diagram helps in understanding the relational structure of the database.

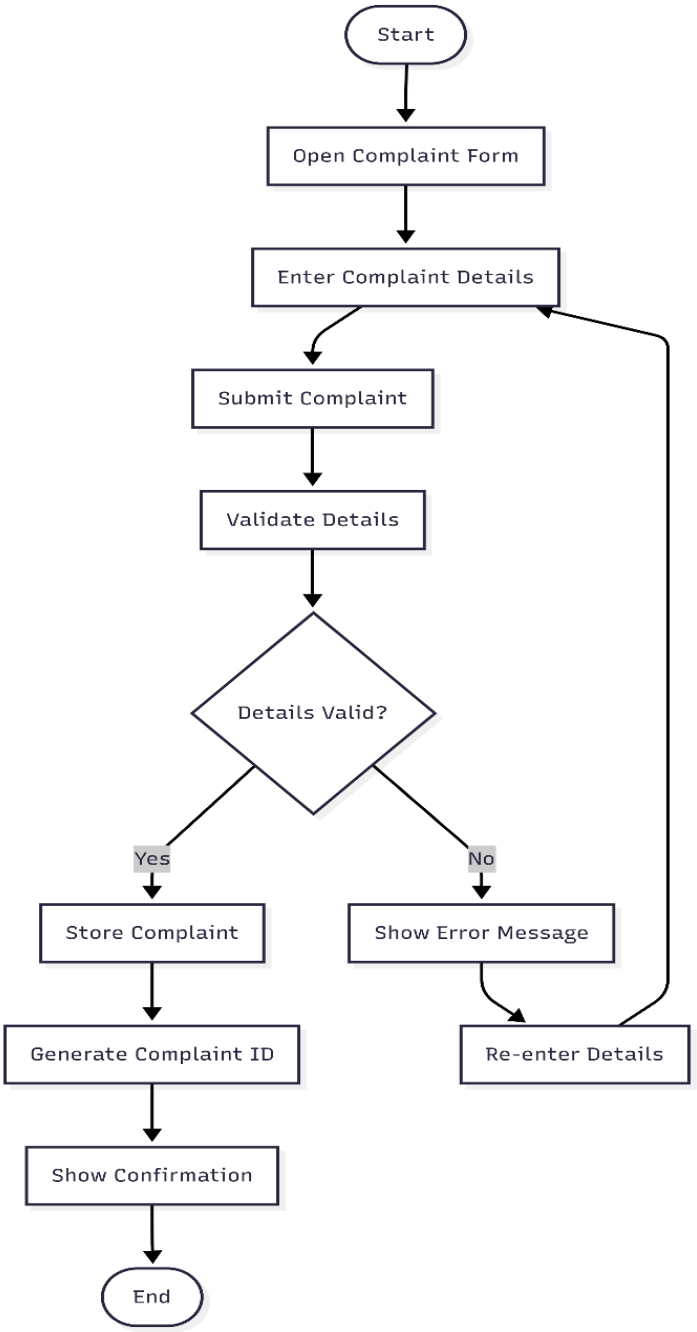
8.2 Use Case Diagram of Water Supply Complaint Management System



• Actors: Admin and User

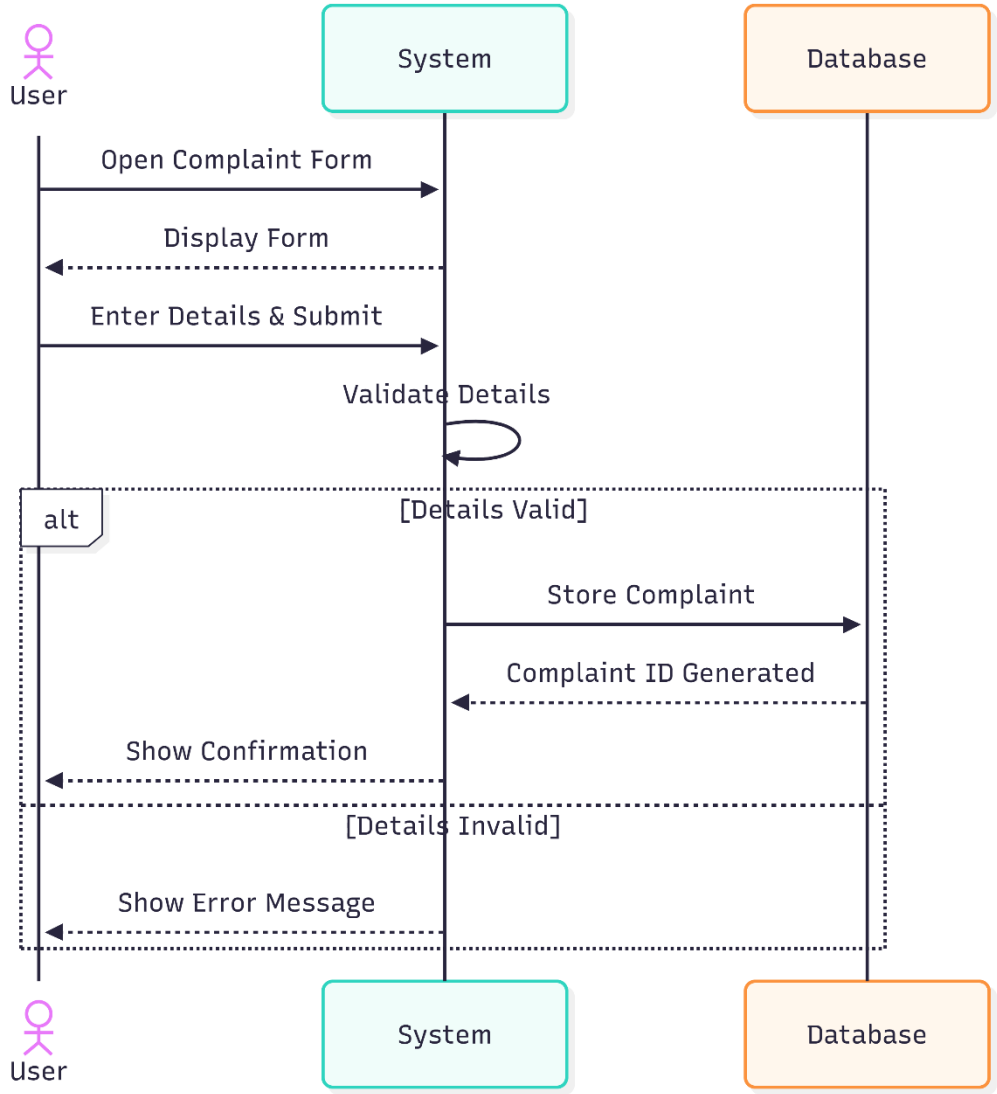
The Use Case Diagram shows how users interact with the system. The User registers complaints and checks status, while the Admin manages complaints and updates their status. It explains the functional behavior of the system from a user perspective.

8.3 Activity Diagram of Water Supply Complaint Management System



The Activity Diagram represents the step-by-step workflow of complaint management. It shows the process starting from complaint registration, validation, storage in database, and status update by the Admin.

8.4 Sequence Diagram of Water Supply Complaint Management System



The Sequence Diagram illustrates the interaction between User, System, and Database. It shows how complaint requests are sent, processed by the system, stored in the database, and confirmed back to the user.

9. SQL IMPLEMENTATION

This section describes the SQL commands used to implement the Water Supply Complaint Management System. SQL is used to create the database, define table structures, insert data, update complaint records, and retrieve meaningful complaint information from the database.

9.1 Database Creation

The database for the system is created using the CREATE DATABASE command. This command initializes a new database where all complaint-related tables are stored. Using a separate database helps in organizing data efficiently and avoids conflicts with other databases.

Example:

```
CREATE DATABASE WaterSupplyDB;
```

9.2 Table Creation

Tables are created using the CREATE TABLE command. Each table is designed to store specific information such as area details, user details, and complaint records. Appropriate data types and constraints are applied to maintain data integrity and establish relationships between tables.

Example:

```
CREATE TABLE User_Details (
   user_id INT PRIMARY KEY AUTO_INCREMENT,
   user_name VARCHAR(50) NOT NULL,
   phone_no VARCHAR(15) UNIQUE,
   address VARCHAR(100),
   area_id INT
);
```

9.3 Data Insertion

Data is inserted into the tables using the INSERT command. This is used to add area records, user information, and complaint details into the database.

Example:

```
INSERT INTO Complaint (user_id, complaint_type, complaint_date, status)
VALUES (1, 'Water Leakage', CURDATE(), 'Pending');
```

9.4 Data Retrieval

Data retrieval is performed using the SELECT command along with WHERE conditions and JOIN operations. These queries help in viewing complaint details, user information, area-wise complaints, and complaint status by combining data from multiple tables.

Example:
```
SELECT u.user_name, a.area_name, c.complaint_type, c.status
FROM User_Details u
JOIN Complaint c ON u.user_id = c.user_id
JOIN Area a ON u.area_id = a.area_id;
```

9.5 Advanced Queries

Advanced SQL queries are used in the Water Supply Complaint Management System to analyze data and generate meaningful information from the database. These queries help in summarizing data, filtering grouped results, and creating reusable query outputs.

GROUP BY

The GROUP BY clause is used to group records that have similar values. In this project, it is mainly used to group complaints based on status or area in order to calculate totals.

Example:
```
SELECT status, COUNT(*)
FROM Complaint
GROUP BY status;
```

HAVING

The HAVING clause is used with GROUP BY to filter grouped data. It allows conditions to be applied on aggregated results such as complaint count.
Example:
```
SELECT area_id, COUNT(*)
FROM User_Details
GROUP BY area_id
HAVING COUNT(*) > 2;
```

SUBQUERIES

Subqueries are queries written inside another SQL query. In this project, subqueries are used to retrieve users who have registered complaints.

Example:
```
SELECT user_name
FROM User_Details
WHERE user_id IN (SELECT user_id FROM Complaint);
```

VIEWS

Views are virtual tables created using SQL queries. They simplify complex queries by storing them as a single logical structure. In this project, views are used to display complaint summaries by combining data from multiple tables.

Example:
```
CREATE VIEW Complaint_View AS
SELECT u.user_name, c.complaint_type, c.status
FROM User_Details u
JOIN Complaint c ON u.user_id = c.user_id;
```

9.6 Transaction Management

Transaction management is used in the Water Supply Complaint Management System to ensure data consistency during important operations such as complaint registration and status updates. Multiple related SQL operations are grouped into a single transaction.
If all operations are executed successfully, the changes are saved using COMMIT. If any error occurs, the transaction is cancelled using ROLLBACK, ensuring that no partial or incorrect data is stored in the database.

The following transaction commands are used in the project:

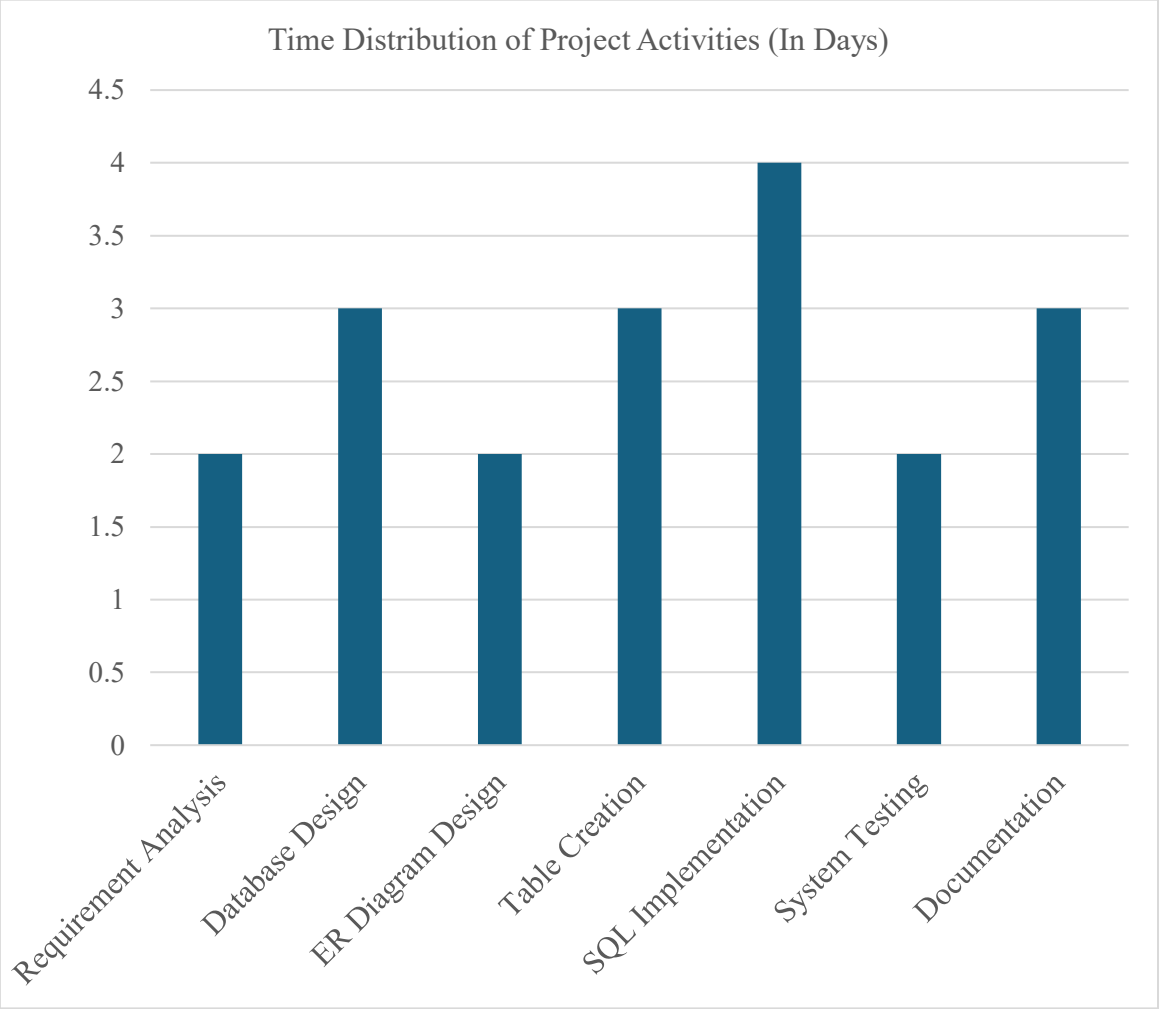
- START TRANSACTION
- COMMIT
- ROLLBACK

Transaction management ensures atomicity and maintains data integrity, making the system reliable and consistent.

Example:
```
START TRANSACTION;
INSERT INTO Complaint (user_id, complaint_type, complaint_date, status)
VALUES (1, 'Low Water Pressure', CURDATE(), 'Pending');
COMMIT;
```

Time Distribution and Work Allocation

The project was divided into structured phases to ensure proper planning and execution. Each activity such as requirement analysis, database design, table creation, SQL implementation, testing, and documentation was allocated time based on its complexity and importance.
The graph below shows the number of days spent on each phase, highlighting that the SQL implementation required the most time among all activities.


Time Distribution of Project Activities (In Days)
4.5
4
3.5
3
2.5
2
1.5
1
0.5
0
Requirement Analysis
Database Design
ER Diagram Design
Table Creation
SQL Implementation
System Testing
Documentation

10. SYSTEM TESTING AND RESULT

System testing was carried out to ensure that the Water Supply Complaint Management System works correctly and produces accurate results. Various SQL queries were executed and verified to check the correctness of database operations, validity of inserted data, proper relationship between tables, and accuracy of generated outputs.

Testing was performed using MySQL Workbench by executing queries step-by-step and verifying the results.

10.1 Query Correctness Testing

All SQL queries such as CREATE, INSERT, SELECT, UPDATE, and transaction-related queries were tested individually.

The queries were executed in proper sequence to verify that:

- Tables were created successfully without any syntax errors
- Data insertion was performed correctly
- Relationships between tables were maintained using foreign keys
- JOIN queries correctly combined data from multiple tables
- Transaction commands (START TRANSACTION, COMMIT, ROLLBACK) executed properly

```
mysql> SHOW TABLES;
+-----------------------+
| Tables_in_watersupply |
+-----------------------+
| area                  |
| complaint             |
| user_details          |
+-----------------------+
3 rows in set (0.047 sec)
```

This output confirms that all required tables (Area, User_Details, Complaint) were created successfully in the database.

```
mysql> SELECT * FROM Area;
+---------+------------+
| area_id | area_name  |
+---------+------------+
|       1 | Panvel     |
|       2 | Kamothe    |
|       3 | Kharghar   |
|       4 | Kalamboli  |
|       5 | New Panvel |
+---------+------------+
5 rows in set (0.006 sec)
```

This output verifies that area records were inserted correctly into the Area table.

```
mysql> SELECT * FROM User_Details;
+---------+----------------+------------+----------------------+---------+
| user_id | user_name      | phone_no   | address              | area_id |
+---------+----------------+------------+----------------------+---------+
|       1 | Amit Patil     | 9000000001 | Sector 1, Panvel     |       1 |
|       2 | Sneha Joshi    | 9000000002 | Sector 5, Panvel     |       1 |
|       3 | Rahul Mehta    | 9000000003 | Sector 10, Kamothe   |       2 |
|       4 | Pooja Sharma   | 9000000004 | Sector 12, Kamothe   |       2 |
|       5 | Vikas Naik     | 9000000005 | Sector 7, Kharghar   |       3 |
|       6 | Neha Verma     | 9000000006 | Sector 15, Kharghar  |       3 |
|       7 | Rohan Desai    | 9000000007 | Sector 3, Kalamboli  |       4 |
|       8 | Priya Nair     | 9000000008 | Sector 8, Kalamboli  |       4 |
|       9 | Sanjay Gupta   | 9000000009 | Sector 2, New Panvel |       5 |
|      10 | Kavita Singh   | 9000000010 | Sector 4, New Panvel |       5 |
|      11 | Arjun Rao      | 9000000011 | Sector 6, Panvel     |       1 |
|      12 | Meera Shah     | 9000000012 | Sector 9, Kamothe    |       2 |
|      13 | Kunal Yadav    | 9000000013 | Sector 11, Kharghar  |       3 |
|      14 | Anjali More    | 9000000014 | Sector 14, Kalamboli |       4 |
|      15 | Deepak Kulkarni | 9000000015 | Sector 16, New Panvel |       5 |
|      16 | Tanvi Salunkhe | 9000000016 | Sector 18, Panvel    |       1 |
|      17 | Nikhil Pawar   | 9000000017 | Sector 19, Kamothe   |       2 |
|      18 | Shruti Iyer    | 9000000018 | Sector 21, Kharghar  |       3 |
|      19 | Manoj Tiwari   | 9000000019 | Sector 22, Kalamboli |       4 |
|      20 | Reshma Khan    | 9000000020 | Sector 23, New Panvel |       5 |
+---------+----------------+------------+----------------------+---------+
20 rows in set (0.014 sec)
```

This output shows that user information was stored accurately in the User_Details table

```
mysql> SELECT * FROM Complaint;
+--------------+---------+--------------------+---------------------+-------------+
| complaint_id | user_id | complaint_type     | complaint_date      | status      |
+--------------+---------+--------------------+---------------------+-------------+
|            1 |       1 | Water Leakage      | 2026-02-21 16:51:38 | Pending     |
|            2 |       2 | No Water Supply    | 2026-02-21 16:51:38 | Resolved    |
|            3 |       3 | Low Water Pressure | 2026-02-21 16:51:38 | Pending     |
|            4 |       4 | Dirty Water        | 2026-02-21 16:51:38 | In Progress |
|            5 |       5 | Pipe Burst         | 2026-02-21 16:51:38 | Pending     |
|            6 |       6 | No Water Supply    | 2026-02-21 16:51:38 | Resolved    |
|            7 |       7 | Water Leakage      | 2026-02-21 16:51:38 | Pending     |
|            8 |       8 | Low Water Pressure | 2026-02-21 16:51:38 | In Progress |
|            9 |       9 | Dirty Water        | 2026-02-21 16:51:38 | Resolved    |
|           10 |      10 | Pipe Burst         | 2026-02-21 16:51:38 | Pending     |
|           11 |      11 | Water Leakage      | 2026-02-21 16:51:38 | Pending     |
|           12 |      12 | No Water Supply    | 2026-02-21 16:51:38 | In Progress |
|           13 |      13 | Low Water Pressure | 2026-02-21 16:51:38 | Resolved    |
|           14 |      14 | Dirty Water        | 2026-02-21 16:51:38 | Pending     |
|           15 |      15 | Pipe Burst         | 2026-02-21 16:51:38 | Resolved    |
|           16 |      16 | Water Leakage      | 2026-02-21 16:51:38 | In Progress |
|           17 |      17 | No Water Supply    | 2026-02-21 16:51:38 | Pending     |
|           18 |      18 | Low Water Pressure | 2026-02-21 16:51:38 | Resolved    |
|           19 |      19 | Dirty Water        | 2026-02-21 16:51:38 | Pending     |
|           20 |      20 | Pipe Burst         | 2026-02-21 16:51:38 | In Progress |
|           21 |       1 | No Water Supply    | 2026-02-21 16:51:38 | Resolved    |
|           22 |       5 | Water Leakage      | 2026-02-21 16:51:38 | Pending     |
|           23 |       8 | Pipe Burst         | 2026-02-21 16:51:38 | Resolved    |
|           24 |      12 | Dirty Water        | 2026-02-21 16:51:38 | Pending     |
|           25 |      16 | Low Water Pressure | 2026-02-21 16:51:38 | In Progress |
+--------------+---------+--------------------+---------------------+-------------+
```

This output represents the registered complaint details including complaint type, complaint date, and status after successful complaint registration.

10.2 Data Validation

Data validation was performed using database constraints to ensure accuracy and integrity of stored information.

The following constraints were verified:
• PRIMARY KEY ensured that each record had a unique identifier
• FOREIGN KEY maintained proper relationships between tables

• NOT NULL prevented important fields from being left empty
• UNIQUE avoided duplicate phone numbers

```
mysql> INSERT INTO User_Details (user_name, phone_no, address, area_id)
    -> VALUES ('Test User','9000000001','Test Address',1);
ERROR 1062 (23000): Duplicate entry '9000000001' for key 'user_details.phone_no'
```

This output shows that the database rejected duplicate phone numbers due to the UNIQUE constraint, ensuring proper data validation and preventing redundancy.

10.3 Output Verification

The system output was verified by executing SELECT and JOIN queries after complaint registration.
The generated outputs were checked to ensure that:

- User name was displayed correctly
- Area name matched the user's registered area
- Complaint type was stored correctly
- Complaint status (Pending / Resolved / In Progress) was displayed accurately
- Summary reports using GROUP BY showed correct complaint counts

```
    -> JOIN Area a ON u.area_id = a.area_id;
+----------------+------------+--------------------+-------------+
| user_name      | area_name  | complaint_type     | status      |
+----------------+------------+--------------------+-------------+
| Amit Patil     | Panvel     | Water Leakage      | Pending     |
| Amit Patil     | Panvel     | No Water Supply    | Resolved    |
| Sneha Joshi    | Panvel     | No Water Supply    | Resolved    |
| Arjun Rao      | Panvel     | Water Leakage      | Pending     |
| Tanvi Salunkhe | Panvel     | Water Leakage      | In Progress |
| Tanvi Salunkhe | Panvel     | Low Water Pressure | In Progress |
| Rahul Mehta    | Kamothe    | Low Water Pressure | Pending     |
| Pooja Sharma   | Kamothe    | Dirty Water        | In Progress |
| Meera Shah     | Kamothe    | No Water Supply    | In Progress |
| Meera Shah     | Kamothe    | Dirty Water        | Pending     |
| Nikhil Pawar   | Kamothe    | No Water Supply    | Pending     |
| Vikas Naik     | Kharghar   | Pipe Burst         | Pending     |
| Vikas Naik     | Kharghar   | Water Leakage      | Pending     |
| Neha Verma     | Kharghar   | No Water Supply    | Resolved    |
| Kunal Yadav    | Kharghar   | Low Water Pressure | Resolved    |
| Shruti Iyer    | Kharghar   | Low Water Pressure | Resolved    |
| Rohan Desai    | Kalamboli  | Water Leakage      | Pending     |
| Priya Nair     | Kalamboli  | Low Water Pressure | In Progress |
| Priya Nair     | Kalamboli  | Pipe Burst         | Resolved    |
| Anjali More    | Kalamboli  | Dirty Water        | Pending     |
| Manoj Tiwari   | Kalamboli  | Dirty Water        | Pending     |
| Sanjay Gupta   | New Panvel | Dirty Water        | Resolved    |
| Kavita Singh   | New Panvel | Pipe Burst         | Pending     |
| Deepak Kulkarni | New Panvel | Pipe Burst         | Resolved    |
| Reshma Khan    | New Panvel | Pipe Burst         | In Progress |
+----------------+------------+--------------------+-------------+
25 rows in set (0.026 sec)
```

This output verifies that the system successfully displayed complaint details by combining data from multiple related tables.

```
    -> GROUP BY status;
+-------------+------------------+
| status      | Total_Complaints |
+-------------+------------------+
| Pending     |               11 |
| Resolved    |                8 |
| In Progress |                6 |
+-------------+------------------+
3 rows in set (0.017 sec)
```

This output represents summary report generation showing the number of complaints based on status.

10.4 Result Analysis

The results obtained from system testing confirm that the database operates correctly and efficiently. All SQL queries were executed successfully without errors. The system was able to store area data, user details, and complaint records accurately.

Constraint validation ensured that duplicate and invalid data entries were restricted. JOIN operations correctly retrieved relational data, and GROUP BY queries successfully generated summary reports.

Overall, the outputs matched the expected results, proving that the Water Supply Complaint Management System meets all functional requirements and works reliably

11. SECURITY, BACKUP AND RECOVERY

Security, backup, and recovery are essential components of any database system. In the Water Supply Complaint Management System, these mechanisms ensure that complaint data is protected from unauthorized access, accidental loss, and system failures.

Proper implementation of security controls and backup procedures improves the reliability and integrity of the database system.

11.1 Security

Security is important in the Water Supply Complaint Management System to protect user and complaint data from unauthorized access.

MySQL provides role-based access control using GRANT and REVOKE commands. These commands allow the administrator to assign or remove specific permissions such as viewing or modifying data.

In addition, database constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, and UNIQUE are used to maintain data integrity and prevent invalid or duplicate entries.

11.2 Backup

Backup is the process of creating a copy of the database to prevent data loss. In this project, database backup can be taken using MySQL utilities to ensure that complaint data remains safe in case of system failure.

11.3 Recovery

Recovery is the process of restoring the database using the backup file. In case of accidental deletion or system crash, the database can be restored to its previous state using the saved backup file

12. FUTURE SCOPE AND CONCLUSION

12.1 Future Scope

The Water Supply Complaint Management System can be further enhanced by adding advanced features and technologies. A web-based or mobile application interface can be developed to allow citizens to register complaints online and track their complaint status in real-time.

Additional features such as SMS or email notifications can be integrated to inform users when their complaint status is updated. An administrative dashboard can also be added to monitor area-wise complaint statistics, resolution rates, and pending complaints.

Advanced reporting and analytics tools may be included to analyze complaint trends, frequently reported issues, and response time performance. These improvements can help the water department make better decisions and improve service quality.

12.2 Conclusion

This project successfully demonstrates the design and implementation of a Water Supply Complaint Management System using MySQL. The system efficiently manages area details, user information, and complaint records while maintaining data integrity through primary keys, foreign keys, and other constraints.

Through this project, practical knowledge of relational database design, SQL queries, JOIN operations, GROUP BY reports, and basic transaction management was gained.

The use of MySQL as an open-source database management system highlights its importance in developing structured, reliable, and scalable database-driven applications for real-world public service management

13. REFERENCES

1. MySQL Official Documentation, Oracle Corporation.
2. Prescribed Database Management System (DBMS) textbooks.
3. Online learning resources related to SQL and MySQL.

14. GLOSSARY

• **DBMS (Database Management System):**
Software used to store, manage, and retrieve data efficiently in the form of databases.
• **SQL (Structured Query Language):**
A standard language used to create, insert, update, delete, and retrieve data from a database.
• **Primary Key:**
A unique identifier for each record in a table that does not allow duplicate or NULL values.
• **Foreign Key:**
A field in a table that creates a relationship with the primary key of another table to maintain data consistency.
• **Transaction:**
A group of SQL operations executed together to ensure data consistency. It follows the principle of either complete success or complete failure.
• **MySQL:**
An open-source relational database management system used to store and manage structured data.

15. APPENDIX / SQL CODE

```
CREATE DATABASE WaterSupply;
USE WaterSupply;

-- Area Table
CREATE TABLE Area (
    area_id INT AUTO_INCREMENT PRIMARY KEY,
    area_name VARCHAR(50) NOT NULL UNIQUE
);

-- User Table
CREATE TABLE User_Details (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    user_name VARCHAR(50) NOT NULL,
    phone_no VARCHAR(15) NOT NULL UNIQUE,
    address VARCHAR(100) NOT NULL,
    area_id INT,
    FOREIGN KEY (area_id) REFERENCES Area(area_id)
);

-- Complaint Table
CREATE TABLE Complaint (
    complaint_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    complaint_type VARCHAR(50) NOT NULL,
    complaint_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) DEFAULT 'Pending',
    FOREIGN KEY (user_id) REFERENCES User_Details(user_id)
);

-- Insert Areas
INSERT INTO Area (area_name) VALUES
('Panvel'),
('Kamothe'),
('Kharghar'),
('Kalamboli'),
('New Panvel');

-- Insert Users (Exact as your database)
INSERT INTO User_Details (user_name, phone_no, address, area_id) VALUES
('Amit Patil','9000000001','Sector 1, Panvel',1),
('Sneha Joshi','9000000002','Sector 5, Panvel',1),
('Rahul Mehta','9000000003','Sector 10, Kamothe',2),
('Pooja Sharma','9000000004','Sector 12, Kamothe',2),
('Vikas Naik','9000000005','Sector 7, Kharghar',3),
('Neha Verma','9000000006','Sector 15, Kharghar',3),
('Rohan Desai','9000000007','Sector 3, Kalamboli',4),
('Priya Nair','9000000008','Sector 8, Kalamboli',4),
('Sanjay Gupta','9000000009','Sector 2, New Panvel',5),
('Kavita Singh','9000000010','Sector 4, New Panvel',5),
```

```
('Arjun Rao','9000000011','Sector 6, Panvel',1),
('Meera Shah','9000000012','Sector 9, Kamothe',2),
('Kunal Yadav','9000000013','Sector 11, Kharghar',3),
('Anjali More','9000000014','Sector 14, Kalamboli',4),
('Deepak Kulkarni','9000000015','Sector 16, New Panvel',5),
('Tanvi Salunkhe','9000000016','Sector 18, Panvel',1),
('Nikhil Pawar','9000000017','Sector 19, Kamothe',2),
('Shruti Iyer','9000000018','Sector 21, Kharghar',3),
('Manoj Tiwari','9000000019','Sector 22, Kalamboli',4),
('Reshma Khan','9000000020','Sector 23, New Panvel',5);

-- Insert Complaints (25 records as per your screenshot)
INSERT INTO Complaint (user_id, complaint_type, status) VALUES
(1,'Water Leakage','Pending'),
(2,'No Water Supply','Resolved'),
(3,'Low Water Pressure','Pending'),
(4,'Dirty Water','In Progress'),
(5,'Pipe Burst','Pending'),
(6,'No Water Supply','Resolved'),
(7,'Water Leakage','Pending'),
(8,'Low Water Pressure','In Progress'),
(9,'Dirty Water','Resolved'),
(10,'Pipe Burst','Pending'),
(11,'Water Leakage','Pending'),
(12,'No Water Supply','In Progress'),
(13,'Low Water Pressure','Resolved'),
(14,'Dirty Water','Pending'),
(15,'Pipe Burst','Resolved'),
(16,'Water Leakage','In Progress'),
(17,'No Water Supply','Pending'),
(18,'Low Water Pressure','Resolved'),
(19,'Dirty Water','Pending'),
(20,'Pipe Burst','In Progress'),
(1,'No Water Supply','Resolved'),
(5,'Water Leakage','Pending'),
(8,'Pipe Burst','Resolved'),
(12,'Dirty Water','Pending'),
(16,'Low Water Pressure','In Progress');

-- JOIN Query
SELECT u.user_name,
       a.area_name,
       c.complaint_type,
       c.status
FROM User_Details u
JOIN Complaint c ON u.user_id = c.user_id
JOIN Area a ON u.area_id = a.area_id;

-- GROUP BY Query
SELECT status, COUNT(*) AS Total_Complaints
FROM Complaint
```

```
GROUP BY status;

-- Transaction Example
START TRANSACTION;

INSERT INTO Complaint (user_id, complaint_type, status)
VALUES (3,'Water Leakage','Pending');

COMMIT;
```